

Biological Data Analysis with R

R & RStudio

Installation Tutorial

Syntax & Operators

What We'll Cover

01

Getting Started

Why R? Installing R and RStudio, and touring the IDE

02

The R Console

Running commands interactively and understanding the prompt

03

R Scripts

Writing, saving, and executing reusable .R files

04

Syntax & Operators

Rules of the language, arithmetic, relational & logical operators

05

Variables & Data Types

Assigning values and R's core data types

06

Hands-on Practice

Guided exercises to build confidence

Why Learn R?

R is a free, open-source programming language purpose-built for statistical computing, data analysis, and graphics — widely used across biology, bioinformatics, and environmental science.



Free & Open Source

No licensing cost — install and use anywhere



Built for Data & Stats

Native support for statistical tests, models & plots



Huge Package Ecosystem

18,000+ CRAN packages incl. Bioconductor for biology



Reproducible Research

Scripts document every analysis step exactly

R vs. Other Tools

Cost	Free	Often paid (SPSS, MATLAB)
Extensibility	18,000+ packages	Limited add-ons
Graphics	Publication-ready	Basic charting
Community	Very large, active	Varies

Tip: RStudio is the IDE that makes writing and running R code far easier.

Step 1: Installing R

R is the engine (the language & interpreter). Download it first from CRAN, the Comprehensive R Archive Network.

1

Visit CRAN

Go to cran.r-project.org
in your browser

2

Choose your OS

Click “Download R for
Windows / macOS /
Linux”

3

Get the installer

Windows: “base” →
“Download R x.x.x for
Windows”

4

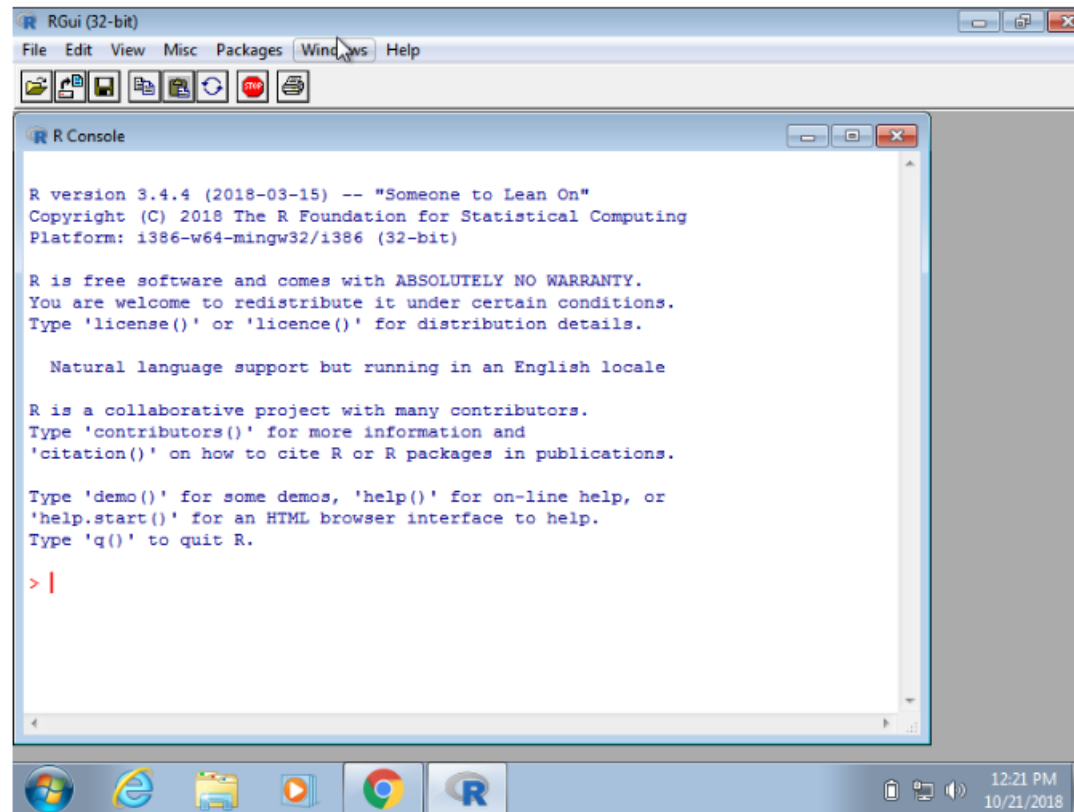
Run & Install

Accept defaults; installs
to Program Files
(Windows)



Good to know: Install R before RStudio. RStudio is only an interface — it needs R installed on your machine to run any code. Re-install R (not RStudio) when you want to upgrade your R version.

Step 1: Installing R



Step 2: Installing RStudio

RStudio Desktop is a free Integrated Development Environment (IDE) that makes writing, running, and debugging R code much easier.

1

Visit

posit.co/download/rstudio-desktop

2

Scroll to Step 2

“Install RStudio Desktop” — free version

3

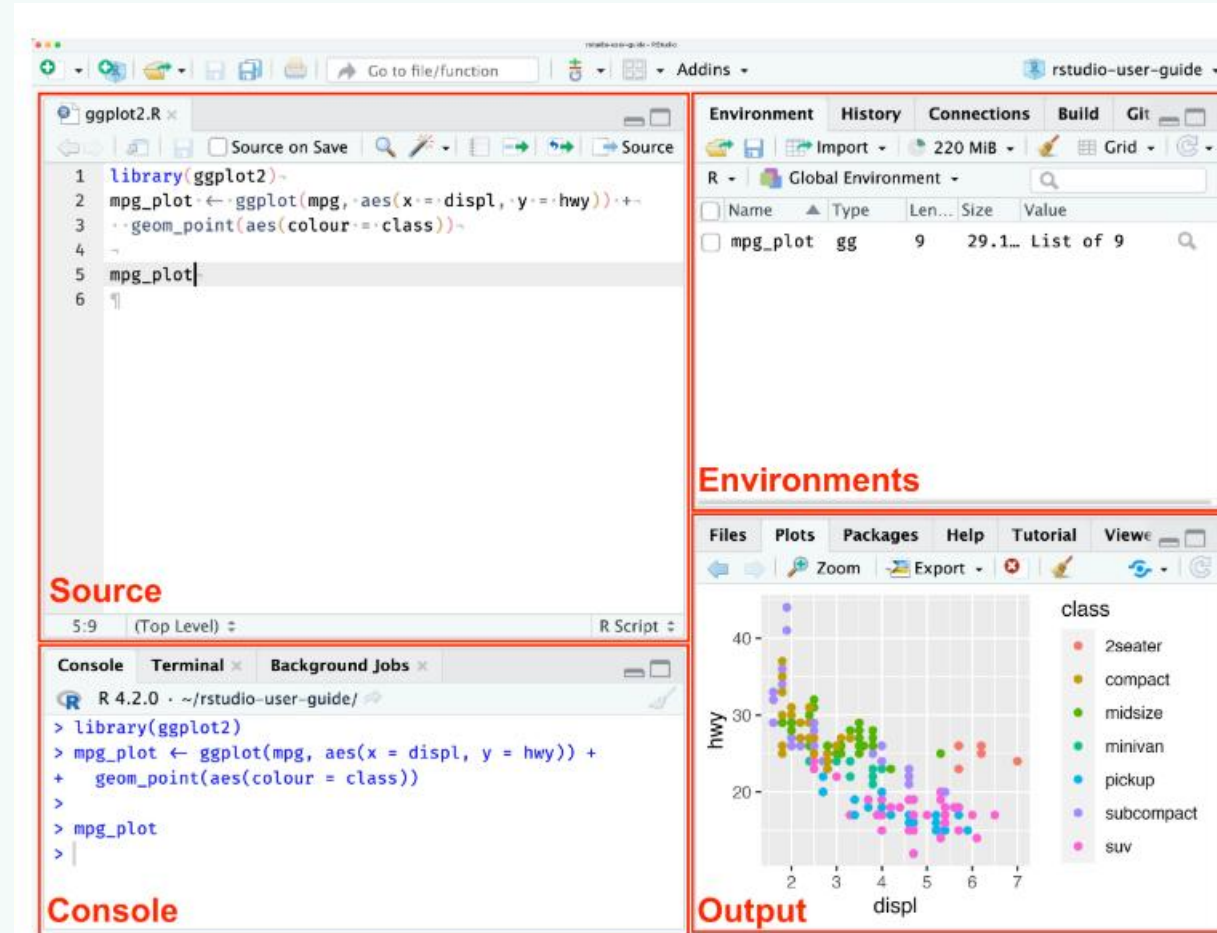
Download installer

It auto-detects your operating system

4

Install & launch

Open RStudio; it auto-locates your R installation



Touring the RStudio Interface

1

Source / Script Editor

Top-left. Where you write, edit, and save .R script files. Click “Run” or press Ctrl+Enter to execute a line.

2

Console

Bottom-left. Where code actually runs; type commands after the > prompt and press Enter to see results instantly.

3

Environment / History

Top-right. Lists every variable and object currently in memory, plus a log of past commands.

4

Files / Plots / Packages / Help

Bottom-right. Browse your project folder, view charts, manage packages, and read documentation.

PART 2

Introduction to R Programming Fundamentals

R Console & Scripts • Basic Syntax • Operators • Variables & Data Types

Initially we will be using R software only (not Rstudio) to familiarize the R programming



```
RGui (32-bit)
File Edit View Misc Packages Windows Help

R Console

R version 3.4.4 (2018-03-15) -- "Someone to Lean On"
Copyright (C) 2018 The R Foundation for Statistical Computing
Platform: i386-w64-mingw32/i386 (32-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> |
```


The R Console & R Scripts

R CONSOLE

```
> 2 + 2  
[1] 4  
>
```

Interactive & immediate.

Type a command after the `>` prompt and press Enter — R runs it right away and shows the result.

Best for: quick checks, exploring data, testing a single line.

R SCRIPT (.R FILE)

```
# my_first_script.R  
x <- 5  
y <- 10  
print(x + y)
```

Saved & reusable.

A plain-text file of R commands you can save, edit, and re-run any time. Run a line with Ctrl+Enter or the whole file with Source.

Best for: full analyses, anything you'll repeat or share.

Basic Syntax Rules

1

Case-sensitive

```
value
```

R treats these as two different objects — spelling & capitalization must match exactly.

2

Comments with

```
# this line is ignored
```

Anything after # on a line is a comment; R does not execute it.

3

Statements

```
x <- 10
```

Each line is usually one statement. A newline typically ends it (no semicolon required).

4

Function calls

```
sum(2, 3, 4)
```

Function name, then arguments inside parentheses, separated by commas.

Operators in R

Arithmetic

+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Exponent
%%	Modulus (remainder)

Relational

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal

Logical

&	AND (vectorized)
	OR (vectorized)
!	NOT
&&	AND (single)
	OR (single)
isTRUE ()	Test for TRUE

Assignment: Storing Values

Assignment operators store a value in a named object so you can reuse it later.



```
x <- 5
```

The preferred, idiomatic R assignment operator (Alt + - shortcut in RStudio)



```
x = 5
```

Also works for assignment, but conventionally reserved for function arguments



```
x <<- 5
```

Global assignment — assigns in the parent/global environment (used sparingly)

Variables in R

A variable is a named container that holds a value in memory. Create one by assigning a value to a name.

Naming Rules

- ✓ Must start with a letter (or a dot not followed by a number)
- ✓ Can contain letters, numbers, dots (.) and underscores (_)
- ✓ Cannot start with a number, e.g. 1x is invalid
- ✓ Case-sensitive: age and Age are different variables
- ✓ Avoid reserved words: TRUE, FALSE, NULL, if, for, function

TRY IT IN THE CONSOLE

```
> name <- "Nitish"
> age <- 30
> age
[1] 30
> age <- age + 1    # update it
> age
[1] 31
```

*A variable's value can change — that's why it's called a “variable.”
Reassigning simply overwrites the old value.*

Basic Data Types

Numeric

`class(): "num"`

Example value

23.5

In code

```
x <- 23.5
```

Integer

`class(): "int"`

Example value

23L

In code

```
x <- 23L
```

Character

`class(): "chr"`

Example value

"DNA"

In code

```
x <- "DNA"
```

Logical

`class(): "logi"`

Example value

TRUE / FALSE

In code

```
x <- TRUE
```

Complex

`class(): "cplx"`

Example value

2+3i

In code

```
x <- 2+3i
```

Check any type with `class(x)` or test it with `is.numeric(x)`, `is.character(x)`, etc.

Hands-On Practice

Open RStudio, create a new script (File → New File → R Script), save it as practice.R, and try each task below.

Task 1

Console warm-up

```
Type 15 * 4 and (18 - 3) / 5 directly into the console.
```

Task 2

Create variables

```
species <- "E. coli"; od600 <- 0.85; is_growing <- TRUE
```

Task 3

Use operators

```
Check if od600 > 0.5 and print species and od600 together  
with paste().
```

Task 4

Check data types

```
Run class() on each variable you created above.
```

Key Takeaways

1 R vs RStudio

R is the language/engine; RStudio is the IDE that makes it easier to use.

2 Console vs Script

Console runs commands instantly; scripts (.R files) save and re-run your work.

3 Syntax matters

R is case-sensitive; use # for comments and <- to assign values.

4 Operators

Arithmetic, relational, and logical operators let you compute and compare.

5 Data types

Numeric, integer, character, logical, and complex — check with class().